

①

Output:

Constructor 10
Constructor 20
Destructor 20
Destructor 10

1. Exact output Sequence

When Demo d1(10), d2(20); is executed

d1 is created first $\rightarrow$ Constructor 10 is printed

d2 is created second $\rightarrow$ Constructor 20 is printed

At end of main(), objects go out of scope in reverse order

2. Analysis of order

Constructor order: d1 $\rightarrow$ d2 (same as declaration order)

* In C++, local/automatic objects are constructed in the order they are declared. Memory is allocated on the stack sequentially

Destructor order: d2 $\rightarrow$ d1 (Reverse of declaration order)

* Destructors are called in LIFO - Last in first out order the object created last is destroyed first

3. Why Destructors are called in Reverse order

$\Rightarrow$ Stack principle: local objects are stored on stack.

Stack works on LIFO, so to maintain proper cleanup, last object must be removed first

$\Rightarrow$ Dependency safety: if second object d2 depends on first object d1, then d2 must be destroyed before d1. If d1 is destroyed first, d2 would be left with a dangling reference

$\Rightarrow$ Resource Management: it ensures that resources are released in the opposite order they were acquired, preventing resource leak and deadlock

1. Constructor Overloading is having more than one constructor in same class with different number or type of parameters

* For product p1; $\rightarrow$ Default Constructor product () is called

* For product p2 (10); $\rightarrow$ Single parameterized Constructor Product (int x) is called

* For product p3 (10, 20); $\rightarrow$ Two parameterized Constructor Product (int x, int y) is called

2. Predicted output

Code:

DefaultOne paramTwo param
if each in new line:

Default
One param
Two param

Explanation:

* P1 matches Constructor with 0 args $\rightarrow$ print Default

* P2 Matches Constructor with 1 int arg $\rightarrow$ print One param

* P3 Matches Constructor with 2 int args $\rightarrow$ print Two param

3. How Compiler Resolves Constructor Overloading
Compiler resolves using Compile-time polymorphism/Static Binding

$\Rightarrow$ Number of Arguments: Compiler counts arguments in object declaration. P1 has 0, P2 has 1, P3 has 2

$\Rightarrow$ Types of Arguments: Compiler checks data type of arguments (int, float, char) if finds exact match

$\Rightarrow$ Best Match Rule: if exact match not found, Compiler tries type conversion/promotion to find closest match

$\Rightarrow$ If not match or ambiguous match $\rightarrow$ Compiler throws error

6
1. How the + operator works for objects?

the function `Complex operator+(Complex c)` overloads the + operator

When we write `c3 = c1 + c2;` Compiler interprets it as `c3 = c1.operator+(c2);`

Working:

* `c1` is the calling object $\rightarrow$ its member are `r` and `i`

* `c2` is passed as arguments `c` $\rightarrow$ its member are `c.r` and `c.i`

* Inside function, it creates a new object: `return Complex(r + c.r, i + c.i);`

* this new object with Summed real and Imaginary parts is returned to `c3`

2. Predicted Result of adding two objects

Code:

```
Complex c1(3, 2);
```

```
Complex c2(1, 4);
```

```
Complex c3 = c1 + c2;
```

Calculation:

$\Rightarrow$ Real part = `c1.r + c2.r` = `3 + 1` = 4

$\Rightarrow$ Imaginary part = `c1.i + c2.i` = `2 + 4` = 6

Result : `c3` will be `4 + 6i`

eg: `Complex(4, 6)`

3. operator overloading vs function-based Addition

operator overloading	Function-based Addition
$c3 = c1 + c2;$	$c3 = c1.add(c2);$
More Intuitive, looks like Normal math	less Intuitive, need to call function explicitly
Give natural Syntax for user-defined types	Syntax is Verbose
Compiler calls operators implicitly	Programmer calls function explicitly
Improves readability and Code elegance	Reduces readability for mathematical operations

Conclusion: Both do the same addition, but operator overloading provides a more natural and readable way to work with objects, like built-in types, making user-defined types behave

1. What problems occur if shallow copy is used

Default Copy Constructor does shallow copy - it copies the pointer value, not the actual data

Problems:

- * Same memory shared: Both P1. medicalHistory and P2. medicalHistory will point to the same memory location

- * Double Deletion / crash: When destructors of P1 and P2 are called, both will try to delete the same pointer this leads to double free error and program crash

- * Dangling pointer: if P1 is deleted, P2 will point to invalid memory

2. How it leads to Data Corruption / Accidental modification

Because both objects share same memory:

- * If insurance staff modified P2 medical history (e.g., adds billing notes), the original hospital's record, P1 will also get automatically modified

- * Example:

Strcpy (P2. medicalHistory, "Diabetes") will overwrite P1 history also

- * This is critical in Hospital system - it causes data corruption, loss of original diagnosis, legal issues, and wrong treatment if one copy is accidentally altered

3. How Deep Copy Ensures Safe Duplication

Deep Copy means Creating a new separate memory block and Copying the actual data

Implementation:

```
Patient (const Patient &P) {  
    medicalHistory = new char[strlen(P.medicalHistory)+1];  
    strcpy(medicalHistory, P.medicalHistory);  
}
```

Benefits:

- ⇒ Separate Memory: P1 and P2 have different pointers pointing to different memory locations with same content
- ⇒ Independent Modification: Changing P2 history does not affect P1
- ⇒ Safe Deletion: Each destructor will delete its own memory - No double deletion, No crash
- ⇒ Data Safety: patient records for referral / Insurance remain isolated and secure

Conclusion:

For any class with dynamic memory (pointers), we must always write our own Copy Constructor and assignment operator to perform deep copy

1. operator overloading vs Traditional function

Traditional function:

Code:

Polynomial $P_3 = P_1.add(P_2)$

Less Intuitive, looks like a function call not math

Operator overloading:

Code:

Polynomial $P_3 = P_1 + P_2$

More Intuitive, natural mathematical notation +. The Code $P_1 + P_2$ Internally calls $P_1.operator+(P_2)$. It Improves readability, makes user-defined type behave like built in type, and simplifies complex expressions like $P_1 + P_2 + P_3$

2. Internal working of Adding polynomials of Different Degrees

Let $P_1 = 3 + 2x + 5x^2 \rightarrow \text{degree } 2 \rightarrow [3, 2, 5]$

$P_2 = 4 + x^3 \rightarrow \text{degree } 3 \rightarrow [4, 0, 0, 1]$

Addition logic inside operator +:

$\Rightarrow$ find max degree: $\text{maxDeg} = \max(P_1.\text{degree}, P_2.\text{degree})$
 $= 3$

$\Rightarrow$ Create result polynomial with maxDeg

$\Rightarrow$ Loop $i=0$ to maxDeg and add Coefficients

result. Coeff[i] = ($i \leq P_1.\text{degree} ? P_1.\text{Coeff}[i] : 0$) + ($i \leq$
 $P_2.\text{Degree} ?$
 $P_2.\text{Coeff}[i] : 0$)

$\Rightarrow$ Results:

$i=0: 3+4=7$

$i=1: 2+0=2$

$i=2: 5+0=5$

$i=3: 0+1=1$

$\Rightarrow [7, 2, 5, 1] \Rightarrow 7 + 2x + 5x^2 + x^3$

3. Memory Handling Issues With Dynamic Arrays

Since Coefficients are stored dynamically $\text{coeff} = \text{new int} [\text{degree} + 1];$

→ Memory leak: If new is used in Constructor but delete [] is not done in destructor, memory leaks

→ Shallow Copy problem: Default Copy Constructor will copy pointer only. Both objects share same array → double delete crash. Solution: Implement deep copy Constructor and assignment operator

→ Dangling pointer: if one object is destroyed, other other objects pointers becomes Invalid

→ out of bounds: Degree mismatch can cause accessing beyond allocated size if not checked

→ Solution: Must Implement:

* Destructor: $\sim \text{polynomial}() \{ \text{delete} [] \text{coeff}; \}$

* Copy Constructor with deep copy

* Operator = Overloading with deep copy

1. Exact ^① output of program

Output

344

If each on New line:

3

4

4

Execution:

* a1, a2, a3 created $\rightarrow$ totalAccounts = 3 $\rightarrow$ prints 3

* Inner block a4 created $\rightarrow$ totalAccounts = 4 $\rightarrow$ prints 4

* a4 goes out of scope, but totalAccounts not decremented

* After block $\rightarrow$ prints 4

2. Why static member retains value Even After a4 Goes out of Scope

$\Rightarrow$ Single copy: Static int totalAccounts is a class-level variable, not object-level. only one copy exists for all objects, stored in static data segment, not on stack

$\Rightarrow$ Lifetime: lifetime of static variable is entire program, not tied to object scope. it is initialized to 0 only once at Program start

$\Rightarrow$ Constructor only Increments: we only Increment in constructor, we did not decrement in destructor. So even when a4 is destroyed, its destructor does not reduce totalAccounts therefore value remains 4

$\Rightarrow$ if we wanted accurate Count of currently existing accounts, we should add destructor

~Accounts()

{ totalAccounts --; }

3. Why `getTotal()` can be called without creating an object and when static is appropriate

* `getTotal()` is a static member function. It does not have this pointer and does not access any non-static member also

* It only accesses static data `totalAccounts` so it belongs to the class, not to any specific object.

* Hence it can be called as `Account::getTotal()` even if no object exists

When static member are appropriate in Real-world System

⇒ Shared Counters: like `totalAccounts`, `totalStudents`, `totalTransactions` - Common to all objects

⇒ Configuration / Constants: Bank Interest rate same for all accounts static float `InterestRate`

State ⇒ Utility / Factory: Functions that don't need object

⇒ Memory Saving: Avoids duplication of same data in every object

1. Exact output Sequence

Constructor 10

Constructor 20

Destructor 20

Destructor 10

2. Analysis of order of Execution

Constructor Execution:

* d1 (10) is created first, so Demo (int a) is called with a=10 it prints Constructor 10

* d2 (20) is created second, so Constructor is called with a=20 it prints Constructor 20

* order of Constructor is same as order of declaration: d1 → d2

Destructor Execution:

* When main() ends, local automatic objects are destroyed

* d2 is destroyed first. it prints Destructor 20

* d1 is destroyed second it prints Destructor 10

* order of destruction is reverse of Constructor
d2 → d1

3. Why Destructors are called in Reverse order

This follows LIFO - Last In First Out principle because objects are stored on the stack

→ Dependency Safety: if d_2 was created after d_1 , d_2 might be using or depending on d_1 . Destroying d_1 first could leave d_2 with a dangling dependency. Reverse order ensures safety.

→ Stack unwinding: In C++, automatic variables are located on stack. Stack always pops in reverse order of push.

→ Resource Management - RAIL: this is core to C++ RAIL principle. it guarantees that resources are released in exact opposite order they were acquired, preventing resource leaks.

1. Exact output

A B C ~C ~B ~A

If each is one newline with endl, it will be:

A
B
C
~C
~B
~A

2. Analysis of order

Constructor chaining - Top to Bottom:
When c obj; is created

⇒ class A Constructor is called first, because
is the root base class prints A

⇒ class B Constructor is called second,
because B Inherits from A prints B

⇒ class C Constructor is called last,
because C is the most derived prints C

order: A → B → C

Destructor chaining - Bottom to top:

When obj goes out of Scope in main()

⇒ class C destructor called first prints

⇒ class B destructor called second prints ~B

⇒ class A destructor called last prints ~A

order: ~C → ~B → ~A

3. why this order

Why Base Constructor first

A derived class object is built on top of a base class object the derived class may use members or functions initialized by the base class so the base must be fully constructed it ensures a stable, initialized foundation

Why Destructor in Reverse order

this is LIFO principle for safety

⇒ If base was destroyed first the derived object would be left operating on destroyed base data - leading to undefined behavior

⇒ By destroying derived first $\sim C$, then its parent $\sim B$, then root $\sim A$, we ensure child releases its own resources before the parent resources it depends on are released

⇒ this guarantees proper stack unwinding in an inheritance hierarchy

1. Why virtual is essential

→ to achieve dynamic dispatch. Shape - can point to any derived object, virtual makes $s \rightarrow \text{area}()$ call the actual object's overridden $\text{area}()$ at runtime, not at compile time

2. If not virtual

Static binding happens Shape - $s = \text{new Circle}()$; $s \rightarrow \text{area}()$ will call Shape :: $\text{area}()$ always, not Circle :: $\text{area}()$ All Shapes will give wrong / 0 area polymorphism fails

3. Extensibility vs performance

Extensible: you can add triangle etc without changing loop for (auto s: Shapes) $s \rightarrow \text{area}()$; open for extension, closed for modification

Trade off: Each class gets vtable and each object gets vptr call becomes $s \rightarrow \text{vptr} \Rightarrow \text{area}$ - one extra pointer lookup, can't be inlined, slight memory + time overhead

Q10 :- Exception Handling in File Processing System.

1). try contains file processing code, while catch handles errors such as

- * File not found

- * Invalid file format

- * Empty file.

2). custom exceptions: custom exception classes give meaningful error messages for different problems, making debugging easier.

3). Advantages:-

- * Separates normal code from error handling.

- * Makes programs easier to maintain.

- * Handles different errors separately

- * Prevents sudden program termination.